Tagging and releases

A SWAT+ version number is a Git tag on the `main` branch. No source file edits are needed to cut a release. The build system reads the tag and projects it into `src/main.f90` and the executable name.

How the version is read

`CMakeLists.txt` runs `git describe --tags` at configure time. If the current commit is a tagged one, that tag is the version. If not, the output is the most recent reachable tag plus the number of commits since and the abbreviated object name (for example, `61.1-1-g92b4413`).

```
execute_process(
  COMMAND git describe --tags
  WORKING_DIRECTORY ${CMAKE_SOURCE_DIR}
  OUTPUT_VARIABLE TAG
  OUTPUT_STRIP_TRAILING_WHITESPACE
)

set(SWAT_VERSION ${TAG})
set(SWATPLUS_EXE "swatplus-${SWAT_VERSION}.exe")
```

The version becomes part of the executable file name. Building at tag `61.0` produces `swatplus-61.0.exe`. Building one commit past tag `61.1` produces `swatplus-61.1-1-g92b4413.exe`.

Listing available tags

```
git tag
git log
```

Example `git log` output with two tags reachable from `HEAD`:

```
commit 92b44138a3fdb0ada27ed86e2a3a378188f70aef (HEAD -> main, origin/main)
Author: od <odavid@colostate.edu>
Date:   Tue Mar 5 21:43:50 2024 -0700

    fixed 'typo'

commit c5ff4b1083c5b320c72d02ec8b728811bed4441a (tag: 61.1)
...

commit df07e3f572adc21834f05c6711df3566e0231c2f (tag: 61.0)
...
```

Building at a specific tag

Check out the tag, then run the normal CMake build.

```
git checkout 61.0
cmake -B build
cmake --build build
```

The executable will be `swatplus-61.0.exe` (or no extension on Linux and macOS). Switch back to the branch head with `git switch main` and rebuild to get the post-tag version string.

How the version reaches the source

The Fortran source uses a template, `src/main.f90.in`. CMake processes it during configure and writes the final `src/main.f90` with substituted values:

```
prog = " SWAT+ @TODAY@        MODULAR Rev @YEAR@.@SWAT_VERSION@"
    write (*,1000)
    open (9003,file='simulation.out')
    write (9003,1000)
1000 format(1x,"                  SWAT+                ",/,             &
   &           "             Revision @SWAT_VERSION@  ",/,             &
   &           "      Soil & Water Assessment Tool    ",/,             &
   &           "@CMAKE_Fortran_COMPILER_ID@ (@CMAKE_Fortran_COMPILER_VERSION@), @ISO@,
    ↪  @CMAKE_HOST_SYSTEM_NAME@",/,             &
   &           "    Program reading . . . executing",/)
```

The substitution block in `CMakeLists.txt`:

```
if (EXISTS "${PROJECT_SOURCE_DIR}/src/main.f90.in")
    string (TIMESTAMP ISO   "%Y-%m-%d %H:%M:%S")
    string (TIMESTAMP TODAY "%b %d %Y")
    string (TIMESTAMP YEAR  "%Y")

    configure_file(
        "${PROJECT_SOURCE_DIR}/src/main.f90.in"
        "${PROJECT_SOURCE_DIR}/src/main.f90"
    )
endif()
```

After `cmake -B build`, `src/main.f90` contains the resolved values:

```
prog = " SWAT+ Mar 04 2024        MODULAR Rev 2024.61.0.0"

    write (*,1000)
    open (9003,file='simulation.out')
    write (9003,1000)
1000 format(1x,"                  SWAT+                ",/,             &
   &           "             Revision 61.0.0  ",/,             &
   &           "      Soil & Water Assessment Tool    ",/,             &
   &           "GNU (11.4.0), 2024-03-04 14:31:18, Linux",/,             &
   &           "    Program reading . . . executing",/)
```

Do not edit `src/main.f90` directly. It is overwritten every build. Edit `src/main.f90.in` instead.

Why tag-based versioning

- The executable's version always matches the tagged source commit it was built from.
- Version propagation into source and into the binary file name happens automatically.
- GitHub builds source archives and release binaries for each tag through the release workflow in `.github/workflows/build.yml`.